

Question 1 — Inventory Database

Monthly Inventory Stock Change

Concepts Tested

- LAG() across ordered months
- Arithmetic difference from previous month
- Date-based ordering using ORDER BY
- Comparing current inventory with previous inventory

Problem Statement

Given a monthly inventory summary, show each month's **closing stock quantity** along with the stock quantity from the previous month and the change in stock.

The stock change should be calculated as:

current month stock – previous month stock

Required Columns

- month_start
- closing_stock
- prev_month_stock
- stock_change

Schema Details

Database: INVENTORY_DB

Table: MonthlyInventory

Column Name	Data Type	Description
month_start	DATE (PK)	First day of the month
closing_stock	INT	Total stock available at the end of the month

Query to be Implemented

Write a **single SQL query** to:

- Order rows by month_start
- Use LAG(closing_stock) to retrieve the previous month's stock
- Calculate stock_change = closing_stock - prev_month_stock
- Output the required columns

Expected Output Format

month_start	closing_stock	prev_month_stock	stock_change
2025-01-01	5000	NULL	NULL
2025-02-01	5400	5000	400
2025-03-01	5100	5400	-300
2025-04-01	5750	5100	650

Question 2 — SmartMart Database

Monthly Sales Revenue Change

Concepts Tested

- LAG() across ordered months
- Comparing current and previous revenue
- Date-based ordering
- Arithmetic calculation using window-function results

Problem Statement

SmartMart management wants to analyze how sales revenue changes from month to month.

Given a monthly sales summary, show each month's **total sales revenue**, the revenue from the previous month, and the change in revenue.

The revenue change should be calculated as:

current month revenue – previous month revenue

Required Columns

- month_start
- total_revenue
- prev_month_revenue
- revenue_change

Schema Details

Database: SMARTMART_DB

Table: MonthlySales

Column Name	Data Type	Description
month_start	DATE (PK)	First day of the month
total_revenue	DECIMAL(12,2)	Total sales revenue for that month

Query to be Implemented

Write a **single SQL query** to:

- Order rows by month_start
- Use LAG(total_revenue) to fetch the previous month's revenue
- Calculate revenue_change = total_revenue - prev_month_revenue
- Output the required columns

Expected Output Format

month_start	total_revenue	prev_month_revenue	revenue_change
2025-01-01	125000.00	NULL	NULL
2025-02-01	138500.00	125000.00	13500.00
2025-03-01	131000.00	138500.00	-7500.00
2025-04-01	152000.00	131000.00	21000.00

Question 3 — AdventureWorks

Monthly Sales Order Value Change

Concepts Tested

- LAG() across ordered months
- Date-based ordering
- Comparing current month with previous month
- Calculated values using window functions
- Working with a transactional database

Problem Statement

The AdventureWorks sales management team wants to monitor changes in monthly sales performance. Using the sales order data, calculate the **total sales amount for each month** and compare it with the previous month.

The sales change should be calculated as:

current month sales – previous month sales

Required Columns

- month_start
- total_sales
- prev_month_sales
- sales_change

Schema Details

Database: AdventureWorks

Use the appropriate sales order table containing the order date and sales amount.

For example, in versions containing the standard AdventureWorks schema, relevant information can be obtained from:

- Sales.SalesOrderHeader
- OrderDate
- TotalDue

Query to be Implemented

Write a **single SQL query** to:

- Group sales by month
- Calculate the total sales amount for each month
- Order the monthly results by month
- Use LAG() to retrieve the previous month's sales
- Calculate:

$\text{sales_change} = \text{total_sales} - \text{prev_month_sales}$

- Output the required columns

Expected Output Format

month_start	total_sales	prev_month_sales	sales_change
2013-01-01	245000.00	NULL	NULL
2013-02-01	271500.00	245000.00	26500.00
2013-03-01	259800.00	271500.00	-11700.00
2013-04-01	298400.00	259800.00	38600.00

Important: The values above are illustrative. Students should generate the actual results from their AdventureWorks database.

Question 4 — Student Database

Monthly Enrollment Change

Concepts Tested

- LAG() across ordered months
- Aggregation before applying a window function
- Date-based ordering
- Difference from previous period

Problem Statement

The academic department wants to monitor changes in student enrollment over time.

Given student enrollment records, calculate the **total number of enrollments for each month**, show the enrollment count from the previous month, and calculate the change.

The enrollment change should be calculated as:

current month enrollments – previous month enrollments

Required Columns

- month_start
- total_enrollments
- prev_month_enrollments
- enrollment_change

Schema Details

Database: STUDENT_DB

Assume the database contains an Enrollments table with information similar to:

Column Name	Data Type	Description
enrollment_id	INT	Unique enrollment
student_id	INT	Student identifier
course_id	INT	Course identifier
enrollment_date	DATE	Date the student enrolled

Query to be Implemented

Write a **single SQL query** to:

- Group enrollment records by month
- Calculate the number of enrollments for each month

- Order the results by month_start
- Use LAG() to retrieve the previous month's enrollment count
- Calculate:

enrollment_change = total_enrollments - prev_month_enrollments

- Output the required columns

Expected Output Format

month_start	total_enrollments	prev_month_enrollments	enrollment_change
2025-01-01	125	NULL	NULL
2025-02-01	148	125	23
2025-03-01	139	148	-9
2025-04-01	172	139	33

Common Pattern for All 4 Questions

These questions deliberately follow the **same structure as your PDF**:

Raw data → Monthly aggregation → LAG() → Previous value → Arithmetic difference

Database	Business Metric	Window Function	Final Calculation
Inventory	Closing Stock	LAG(closing_stock)	Current – Previous
SmartMart	Sales Revenue	LAG(total_revenue)	Current – Previous
AdventureWorks	Sales Amount	LAG(total_sales)	Current – Previous
Student	Enrollments	LAG(total_enrollments)	Current – Previous